EMBEDDED COMPUTING SYSTEMS EEME30001

Week 7:

Interrupts (Architecture, Functions, and Configuration)



University of BRISTOL

Interrupt Concept

What is an Interrupt ?

An interrupt is a signal that can cause the CPU to temporarily pause its current task and run an Interrupt Service Routine (ISR), though not all interrupts immediately halt the CPU. This process is more efficient than register polling, which continuously checks device status and wastes CPU resources.

Interrupts optimise CPU utilisation by allowing the CPU to perform other tasks until the peripheral requires attention. Some interrupts can be delayed or ignored based on priority, while others, like non-maskable interrupts (NMIs), always take precedence.

Interrupt concept

What is an Interrupt ?

Since interrupts are not in sync with the main program, careful management is needed to prevent conflicts over shared resources (like global variables or hardware).

Interrupts reach the CPU through **flag registers**, which store them until they are processed, and **enable registers**, which control which interrupts are allowed to reach the CPU.

When an interrupt reaches the CPU, it gets the address of the correct Interrupt Service Routine (ISR) from the **vector table**, a memory structure that stores ISR addresses.

The CPU then uses this address to execute the ISR (quickly), ensuring each interrupt is handled correctly, which helps maintain system stability and performance.

Interrupt Architecture

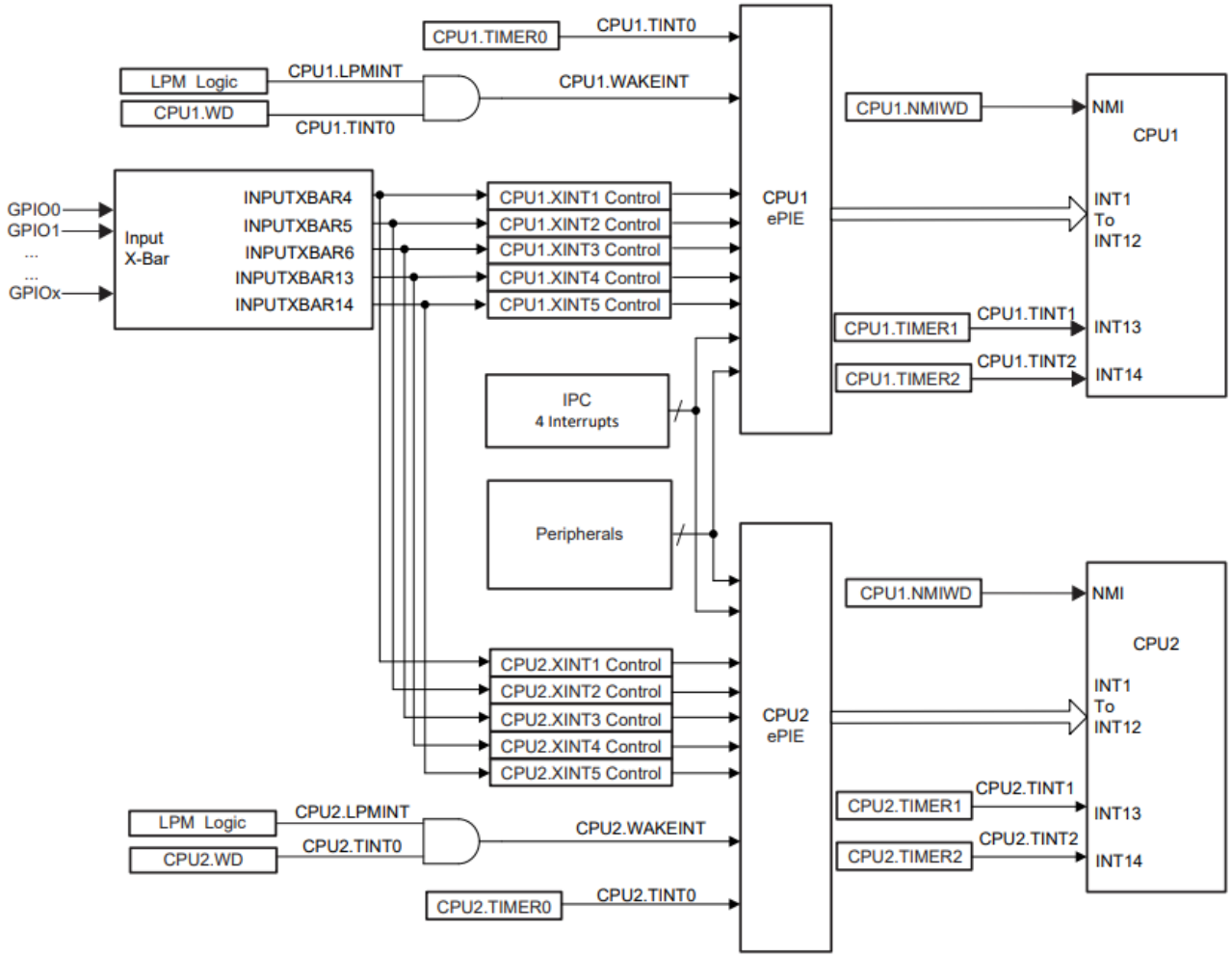
The C28x CPU in the C28379D has fourteen interrupt lines. Two of these (INT13 and INT14) are directly connected to CPU timers, while the other twelve connect to peripheral interrupts through the Peripheral Interrupt Expansion (PIE) module. The PIE can combine up to sixteen peripheral interrupts into each CPU line and gives each interrupt its own ISR. The mapping of peripherals to PIE groups is fixed and cannot be changed, ensuring correct handling and prioritisation of interrupts.

These mappings are predefined by the microcontroller's architecture and documented in the technical manual to ensure efficient interrupt handling.

See next slide

Interrupt Architecture

Figure 3-1. Device Interrupt Architecture



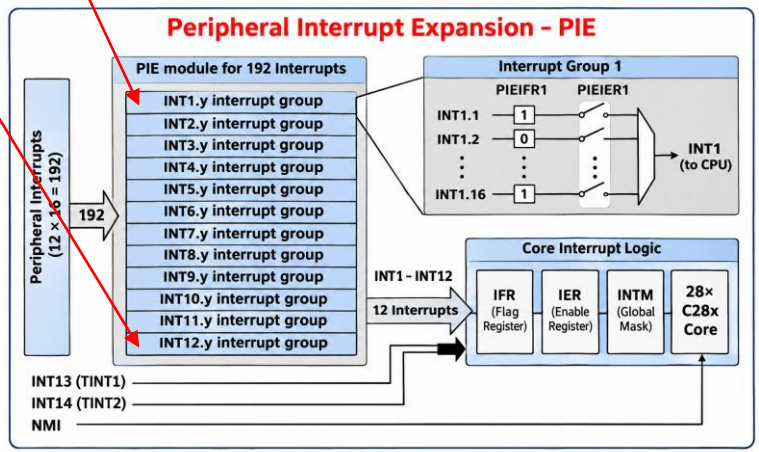
C28379d Functional Block Diagram

Table 3-2. PIE Channel Mapping

PIE Group (CPU INTx)	PIE Channel (1 – 16)															
	Ch. 1	Ch. 2	Ch. 3	Ch. 4	Ch. 5	Ch. 6	Ch. 7	Ch. 8	Ch. 9	Ch. 10	Ch. 11	Ch. 12	Ch. 13	Ch. 14	Ch. 15	Ch. 16
Group 1 (INT1)	ADCA1	ADCB1	ADCC1	XINT1	XINT2	ADCD1	TIMER0	WAKE	–	–	–	–	IPC0	IPC1	IPC2	IPC3
Group 2 (INT2)	EPWM1 TZ	EPWM2 TZ	EPWM3 TZ	EPWM4 TZ	EPWM5 TZ	EPWM6 TZ	EPWM7 TZ	EPWM8 TZ	EPWM9 TZ	EPWM10 TZ	EPWM11 TZ	EPWM12 TZ	–	–	–	–
Group 3 (INT3)	EPWM1	EPWM2	EPWM3	EPWM4	EPWM5	EPWM6	EPWM7	EPWM8	EPWM9	EPWM10	EPWM11	EPWM12	–	–	–	–
Group 4 (INT4)	ECAP1	ECAP2	ECAP3	ECAP4	ECAP5	ECAP6	–	–	–	–	–	–	–	–	–	–
Group 5 (INT5)	EQEP1	EQEP2	EQEP3	–	CLB1	CLB2	CLB3	CLB4	SD1	SD2	–	–	–	–	–	–
Group 6 (INT6)	SPIA_RX	SPIA_TX	SPIB_RX	SPIB_TX	MCBSPA RX	MCBSPA TX	MCBSPB RX	MCBSPB TX	SPIC_RX	SPIC_TX	–	–	–	–	–	–
Group 7 (INT7)	DMA_CH1	DMA_CH2	DMA_CH3	DMA_CH4	DMA_CH6	DMA_CH6	–	–	–	–	–	–	–	–	–	–
Group 8 (INT8)	I2CA	I2CA FIFO	I2CB	I2CB FIFO	SCIC_RX	SCIC_TX	SCID_RX	SCID_TX	–	–	–	–	–	–	UPPA (CPU1 only)	–
Group 9 (INT9)	SCIA_RX	SCIA_TX	SCIB_RX	SCIB_TX	CANA_0	CANA_1	CANB_0	CANB_1	–	–	–	–	–	–	USBA (CPU1 only)	–
Group 10 (INT10)	ADCA EVT	ADCA2	ADCA3	ADCA4	ADCB_EVT	ADCB2	ADCB3	ADCB4	ADCC_EVT	ADCC2	ADCC3	ADCC4	ADCD_EVT	ADCD2	ADCD3	ADCD4
Group 11 (INT11)	CLA1_1	CLA1_2	CLA1_3	CLA1_4	CLA1_5	CLA1_6	CLA1_7	CLA1_8	–	–	–	–	–	–	–	–
Group 12 (INT12)	XINT3	XINT4	XINT5	–	–	VCU	FPU_ OVERFLOW	FPU_ UNDERFLOW	EMIF_ ERROR	RAM_COR RECTABLE _ERROR	FLASH_COR RECTABLE _ERROR	RAM_ACCESS VIOLATION	SYS_PLL SLIP	AUX_PLL SLIP	CLA OVERFLOW	CLA UNDERFLOW

Note: Cells marked "–" are Reserved.

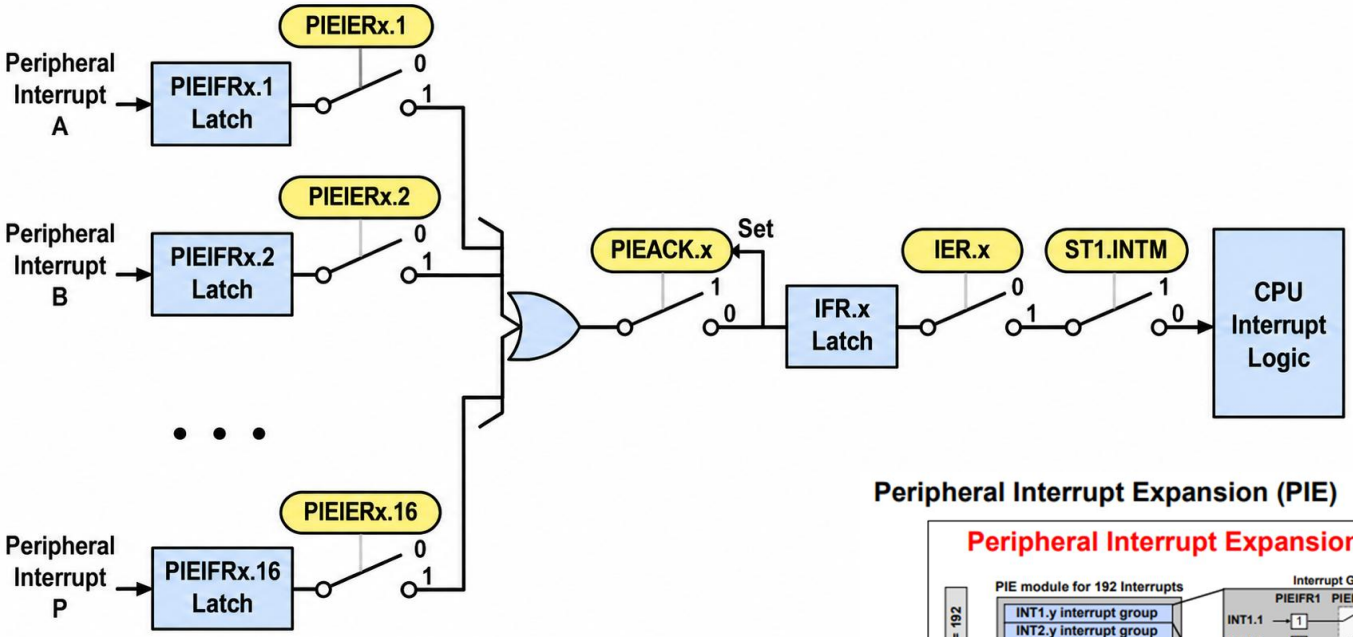
Peripheral Interrupt Expansion (PIE)



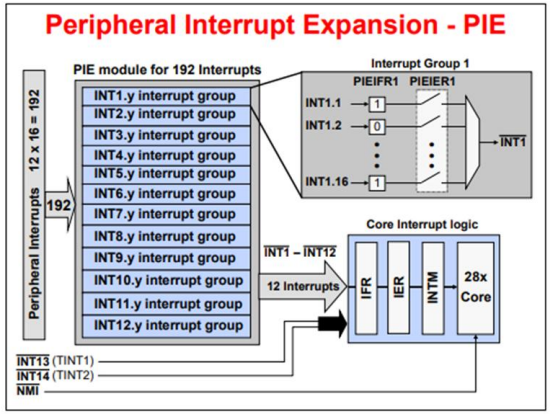
Interrupt Propagation Path

Interrupts reach the CPU through flag registers, which store them until they are processed, and enable registers, which control which interrupts are allowed to reach the CPU.

Figure 3-2. Interrupt Propagation Path



Peripheral Interrupt Expansion (PIE)

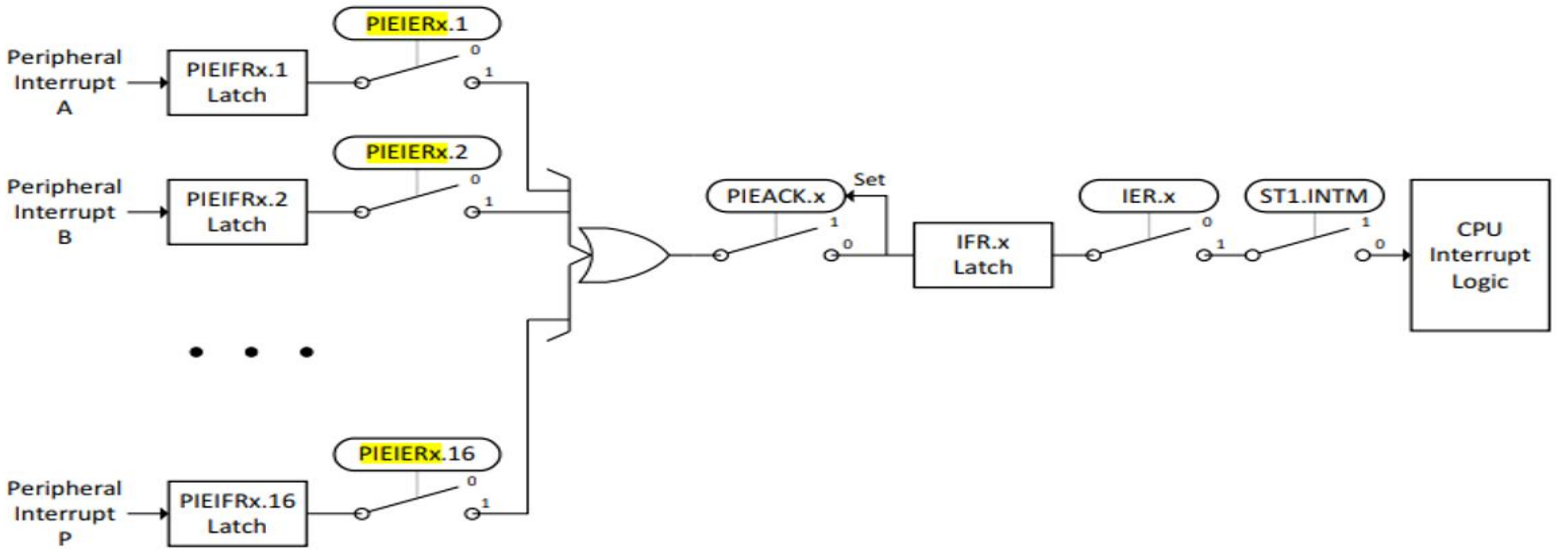


For example, for PIE Group 1:

- Peripheral Interrupt A → INT1.1 (ADCA1)
- Peripheral Interrupt B → INT1.2 (ADCB1)
- Peripheral Interrupt P → INT1.16 (IPC3)

Interrupt Propagation Path

Figure 3-2. Interrupt Propagation Path



1 Interrupt request

A peripheral request sets its PIEIFRx.y flag.

2 Interrupt selection

PIEIERx.y enables its channel; PIEACKx controls group forwarding.

3 Interrupt acceptance

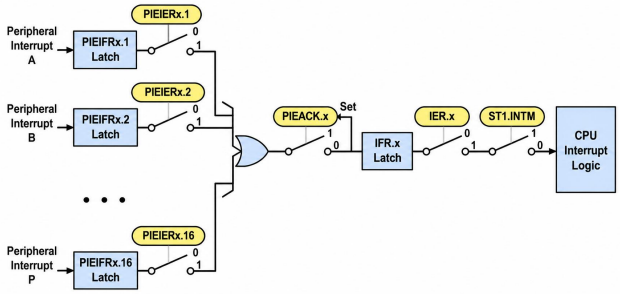
The selected PIEIFR flag clears in hardware when the CPU accepts the interrupt.

Interrupt Propagation Path

The sequence of events when a peripheral generates an interrupt (on PIE group x, channel y) is as follows:

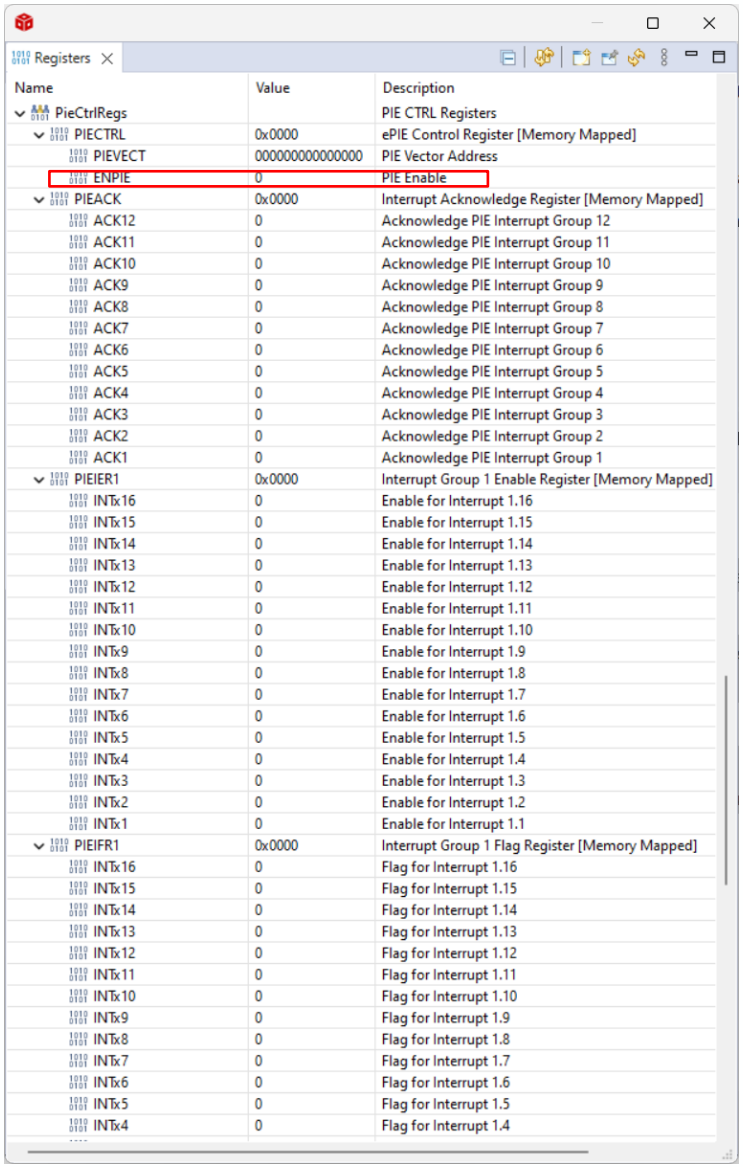
1. The interrupt is latched in PIEIFRx.y.
2. If PIEIERx.y is set, the interrupt propagates.
3. If PIEACK.x is clear, the interrupt propagates, and PIEACK.x is set.
4. The interrupt is latched in IFR.x.
5. If IER.x is set, the interrupt propagates.
6. If INTM is clear, the CPU receives the interrupt.
7. Any instructions in the D2 or later stage of the pipeline are run to completion. Instructions in earlier stages are flushed
8. The CPU saves its context on the stack.
9. On interrupt entry, the CPU **sets INTM = 1** and saves context (PC, ST1, ST0). **IER is unchanged.** The **group flag in IFR.x** is cleared by hardware as the ISR vector is taken, and the **PIEIFR.x.y** bit is cleared by the PIE at that moment.
10. The CPU branches to the ISR.

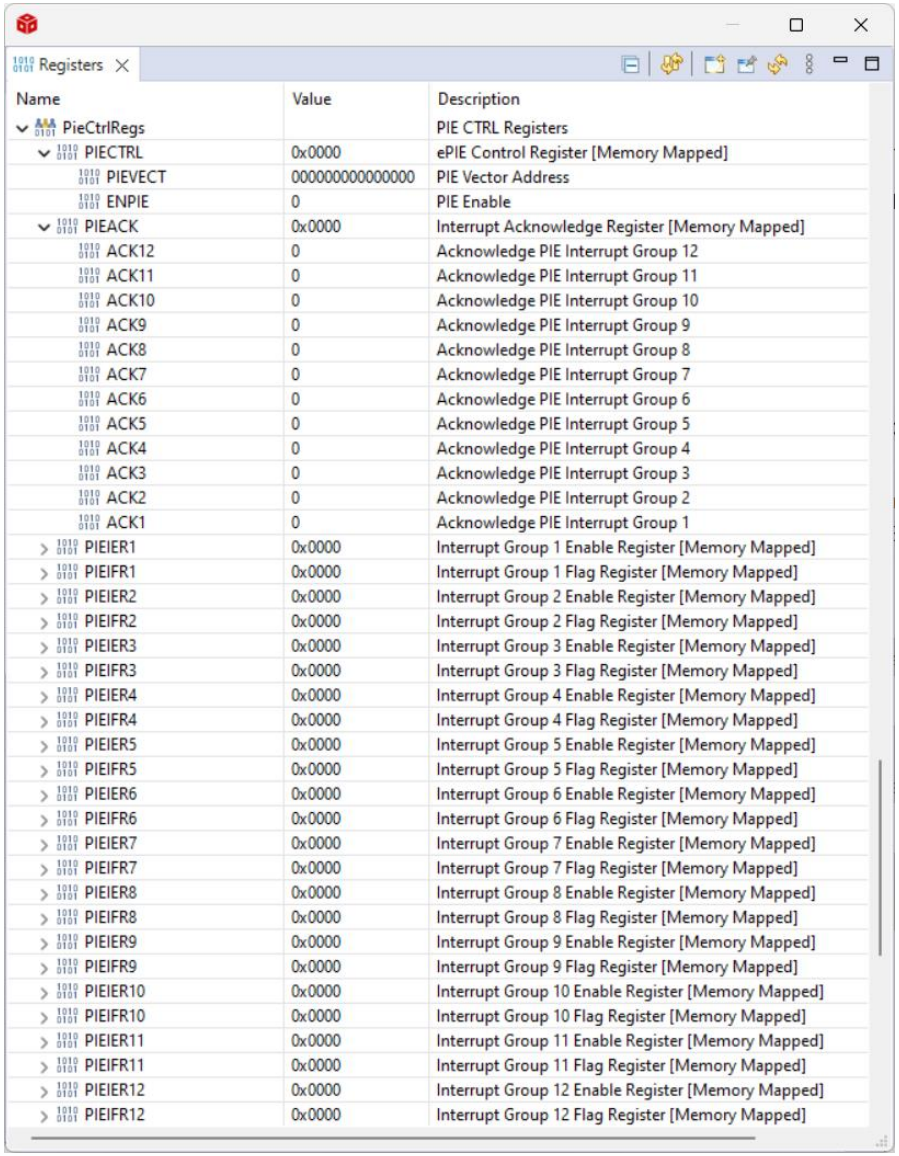
Figure 3-2. Interrupt Propagation Path



The minimum interrupt latency is 14 SYSCLK cycles, but additional delays can occur due to wait states or external interrupt synchronisation.

Configuring and using Interrupts





Configuring and using Interrupts

Enable the PIE by setting the ENPIE bit in the PIECTRL register.

Before using any PIE interrupts (e.g., ADC, ePWM, SCI, I2C…), you must **enable the PIE block** by setting the ENPIE bit in PIECTRL

PIECTRL Register Structure

Bit	Name	Description
15	ENPIE	**Enable PIE**: Enables the PIE when set to 1. Disables the PIE when cleared (0).
14-1	Reserved	Reserved for future use, should be set to 0.
0	PIEVECT	**PIE Vector Fetch Control**: Controls how vectors are fetched.

```
EALLOW;          // Enable protected register

PieCtrlRegs.PIECTRL.bit.ENPIE = 1;    // Enable the PIE module

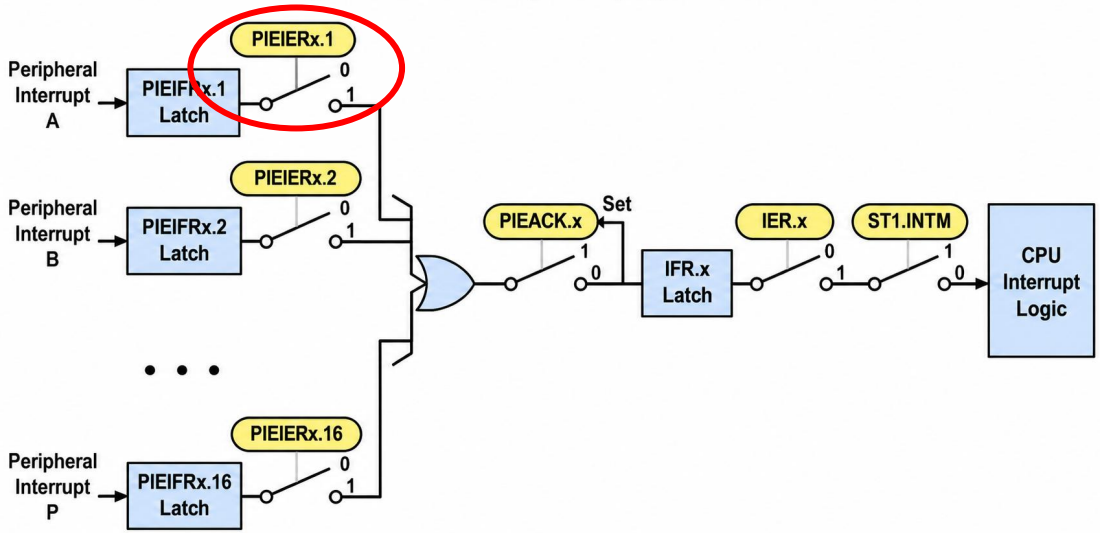
EDIS;                        // Disable protected register access
```

Configuring and using Interrupts

Set the appropriate PIEIERx bit for each interrupt.

```
PieCtrlRegs.PIEIER1.bit.INTx1 = 1;     // Enable PIE group 1, interrupt 1
```

Figure 3-2. Interrupt Propagation Path

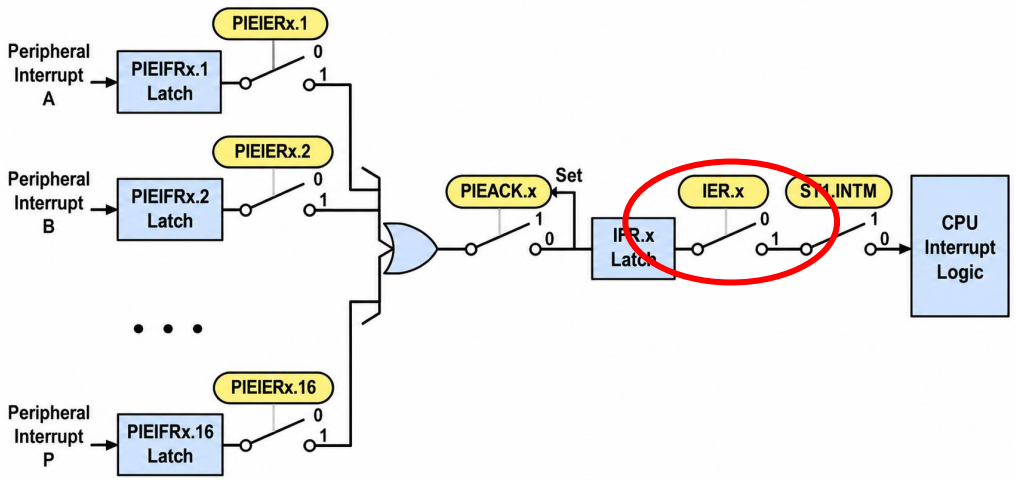


Configuring and using Interrupts

Set the CPU IER bit for any PIE group containing enabled interrupts

```
// Enable CPU interrupt groups 1 and 2
IER |= (1U << 0);  // CPU INT1: PIE Group 1
IER |= (1U << 1);  // CPU INT2: PIE Group 2
```

Figure 3-2. Interrupt Propagation Path



Enable the interrupt in the peripheral.

Enable the interrupt in the peripheral.

Example 1: Enabling the Timer Peripheral (TIMER0):

```
EALLOW;   // Enable access to protected registers

    // Enable the interrupt for TIMER0
    CpuTimer0Regs.TCR.bit.TIE = 1;  // Set the TIE (Timer Interrupt Enable) bit

    EDIS;     // Disable access to protected registers
```

CpuTimer1Regs		CPUTIMER Registers
TIM	0x17D78400	CPU-Timer, Counter Register [Memory Mapped]
PRD	0x17D78400	CPU-Timer, Period Register [Memory Mapped]
TCR	0x4011	CPU-Timer, Control Register [Memory Mapped]
TIF	0	CPU-Timer Interrupt Flag.
TIE	1	CPU-Timer Interrupt Enable.
FREE	0	Emulation modes
SOFT	0	Emulation modes
TRB	0	Timer reload
TSS	1	CPU-Timer stop status bit.
TPR	0x0000	CPU-Timer, Prescale Register [Memory Mapped]
TPRH	0x0000	CPU-Timer, Prescale Register High [Memory Mapped]

Configuring and using Interrupts

Enable the interrupt in the peripheral.

Example 2: Enabling Interrupt in an ADC Peripheral (ADC Interrupt)

```
    EALLOW;   // Enable access to protected registers

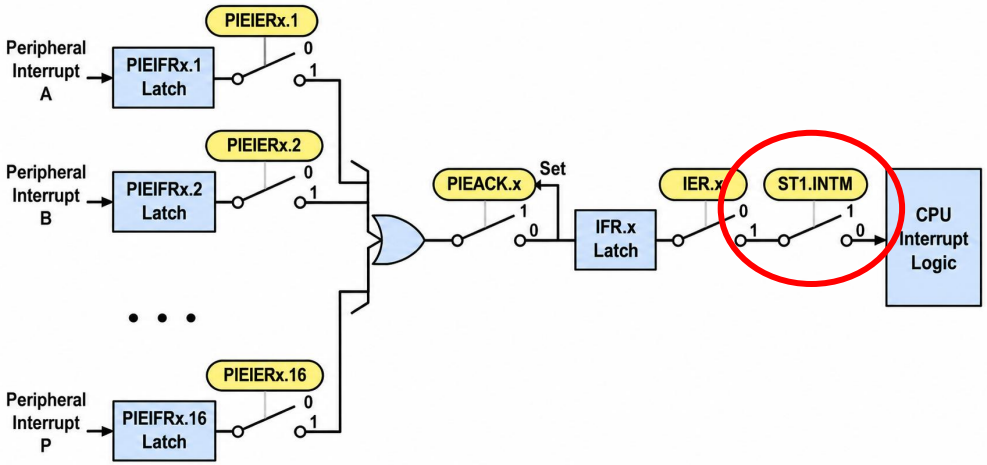
    // Enable the interrupt for ADC
    AdcaRegs.ADCINTSEL1N2.bit.INT1SEL = 1;  // Select ADC EOC source for ADCINT1
    AdcaRegs.ADCINTSEL1N2.bit.INT1E = 1;    // Enable interrupt 1

    EDIS;     // Disable access to protected registers
```

Configuring and using Interrupts

Enable interrupts globally (EINT or CLRC INTM).

Figure 3-2. Interrupt Propagation Path

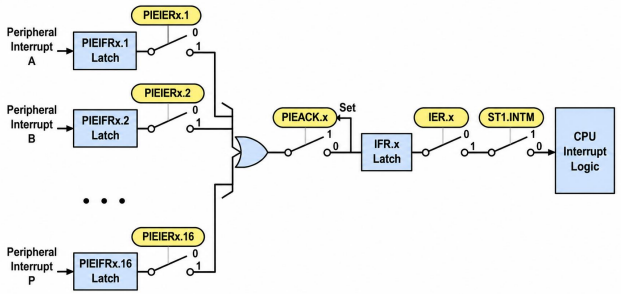


Action	Macro	DriverLib Function	Inline Assembly
Enable global interrupts	EINT;	Interrupt_enableMaster();	__asm(" CLRC INTM");
Disable global interrupts	DINT;	Interrupt_disableMaster();	__asm(" SETC INTM");

PIE Interrupt Acknowledgement in an ISR

Write the ISR vector for each interrupt to the PIE vector table.

Figure 3-2. Interrupt Propagation Path




```
//
// cpuTimer0ISR - ISR for CPU Timer 0
//
__interrupt void cpuTimer0ISR(void)
{
//    timer0= Timer_getTimerCount(CPUTIMER0_BASE);

   cpuTimer0IntCount++;

   // Acknowledge this interrupt to receive more interrupts from group 1
   Interrupt_clearACKGroup(INTERRUPT_ACK_GROUP1);
}

//
// cpuTimer1ISR - ISR for CPU Timer 1
//
__interrupt void cpuTimer1ISR(void)
{
   cpuTimer1IntCount++;

   // Timer 1 uses CPU INT13 directly: no PIE acknowledgement required.
}
```

Why is PIE Acknowledgement Needed?

When a PIE interrupt is accepted, its group becomes busy.

To allow another interrupt from the same group to be forwarded, acknowledge the group.

Use Interrupt_clearACKGroup() in the ISR.

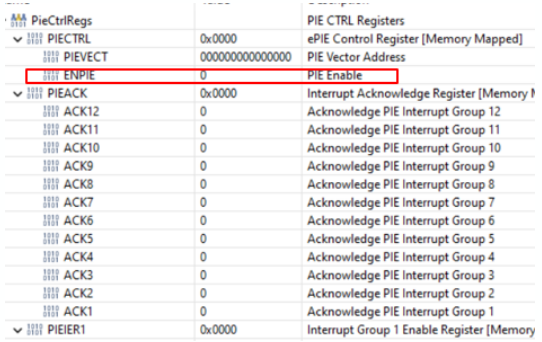
Note: The selected PIEIFR flag normally clears in hardware on interrupt acceptance. PIEACK releases the group busy state.

PIE Group Acknowledgement: Direct Register Access vs DriverLib.

Using DriverLib:

```
    // Acknowledge interrupt group

PieCtrlRegs.PIEACK.all = PIEACK_GROUP3;
```



PieCtrlRegs		PIE CTRL Registers
PIECTRL	0x0000	ePIE Control Register [Memory Mapped]
PIEVECT	0000000000000000	PIE Vector Address
ENPIE	0	PIE Enable
PIEACK	0x0000	Interrupt Acknowledge Register [Memory M
ACK12	0	Acknowledge PIE Interrupt Group 12
ACK11	0	Acknowledge PIE Interrupt Group 11
ACK10	0	Acknowledge PIE Interrupt Group 10
ACK9	0	Acknowledge PIE Interrupt Group 9
ACK8	0	Acknowledge PIE Interrupt Group 8
ACK7	0	Acknowledge PIE Interrupt Group 7
ACK6	0	Acknowledge PIE Interrupt Group 6
ACK5	0	Acknowledge PIE Interrupt Group 5
ACK4	0	Acknowledge PIE Interrupt Group 4
ACK3	0	Acknowledge PIE Interrupt Group 3
ACK2	0	Acknowledge PIE Interrupt Group 2
ACK1	0	Acknowledge PIE Interrupt Group 1
PIEIER1	0x0000	Interrupt Group 1 Enable Register [Memory

- When the CPU accepts an interrupt from PIE Group 3, the group becomes busy, preventing further Group 3 requests from being forwarded.
- Writing 1 to PIEACK.ACK3 releases the busy state, allowing subsequent Group 3 interrupts to be forwarded.

Equivalent direct register access:

```
    // Acknowledge interrupt group

    Interrupt_clearACKGroup(INTERRUPT_ACK_GROUP3);
```